

# Compiladores

## Conceptos Introductorios



# 1. ¿Qué es un Compilador?

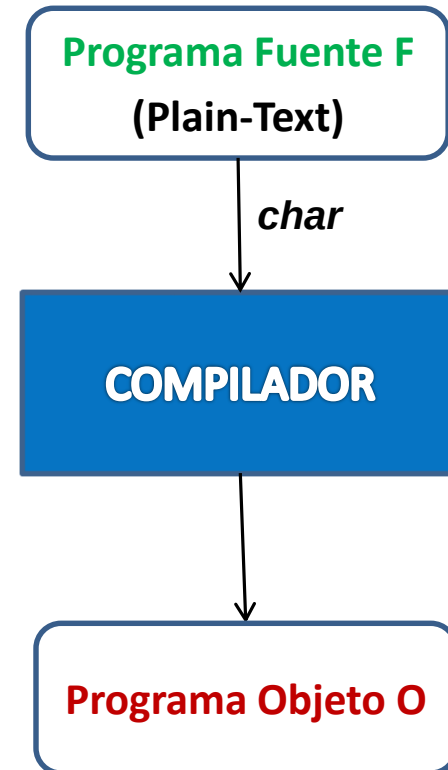
A grandes rasgos un **Compilador** es un software que lee un programa **F** escrito en un lenguaje, el **lenguaje fuente**, y lo traduce a un programa equivalente **O**, en otro lenguaje, el **lenguaje objeto**.

*Como parte importante de este proceso de traducción, el compilador informa a su usuario de la presencia de **errores** en el **programa fuente**.*

# 1. ¿Qué es un Compilador?

El **Programa Fuente\***, mayormente está escrito en Modo Texto (Plain-Text), por tanto:

El **Compilador** lee caracter (char) a caracter el contenido del **Programa Fuente F**, que luego de un proceso de traducción genera un **Programa Objeto\* O**, equivalente a F.



Siempre se debe recordar que un **Compilador** NO CORRE (run) al Programa Objeto. Su trabajo solo se limita a realizar la **TRADUCCIÓN** de F a O.

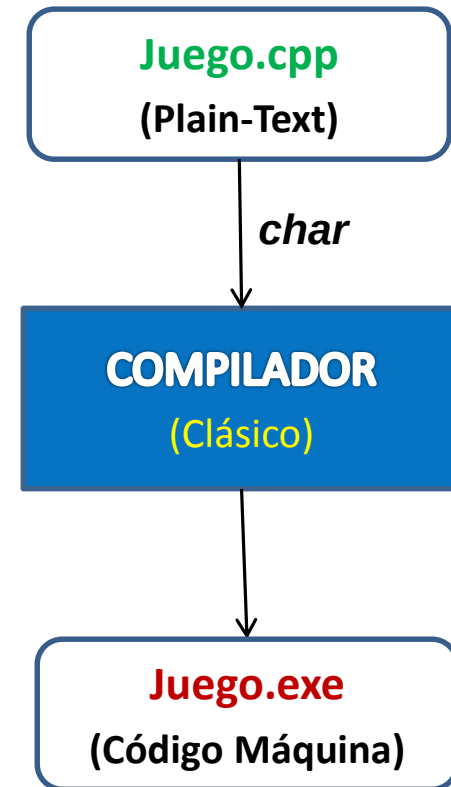
\*En inglés: Programa Fuente = **Source Program**, Programa Objeto = **Target Program**

# 1.1. Compilador Clásico y No-Clásico

Desde sus inicios, los Compiladores fueron creados para leer **Programas Fuentes** escritos en Plain-Text y generar **Programas Objetos** stand-alone de Código Máquina absoluto, a los cuales le llamamos “ejecutables”.

A este tipo de Compiladores, se les llama **Compiladores Clásicos**.

Por ejemplo, en el Diagrama de la derecha, se observa un **Compilador Clásico** que lee un Programa Fuente de C++ y lo convierte a un archivo “ejecutable” en el formato de Windows.

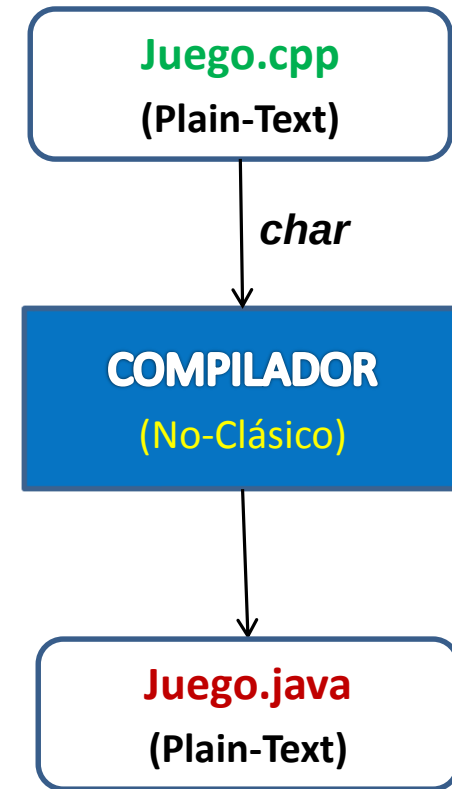


# 1.1. Compilador Clásico y No-Clásico

Sin embargo, hoy en día es posible construir **Compiladores No-Clásicos**, que traduzcan código de de C++ a JAVA, de Delphi-Pascal a C++, etc.

Recuerde que la [definición de compilador](#) no limita el lenguaje del programa objeto.

Por ejemplo, en el Diagrama mostrado, el **Compilador** traduce un Programa Fuente de C++ a un Programa de Java.



Resumiendo:

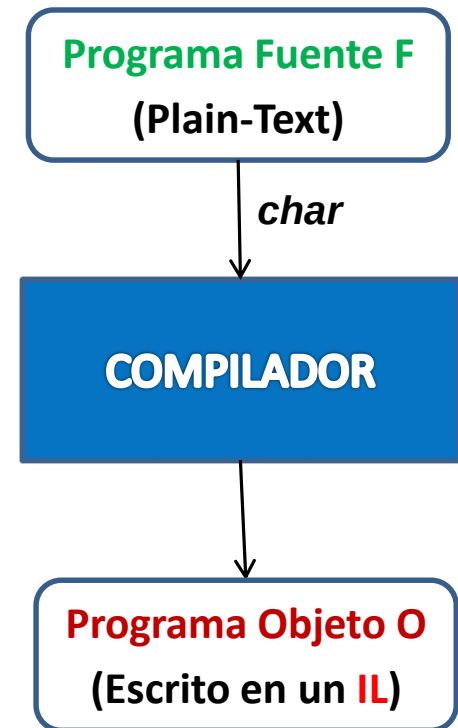
**Compilador Clásico** = Programa Fuente (Texto) → Código Máquina (.exe)

**Compilador No-Clásico** = Programa Fuente (Texto) → Código en otro lenguaje (no-máquina)

## 1.2. Compilación a Código Intermedio

Es más, la “moda” actual, es el diseño de lenguajes de programación, cuyo Compilador traduce el **Programa Fuente F** a un **Programa Objeto** escrito en un **Lenguaje Intermedio\*** (IL), con el objetivo de que éste último sea un código portable.

Posteriormente un programa, llamado **intérprete** de la **VM**, se encargará de la interpretación (run) del **Código Intermedio\*\***.



\* Un **Lenguaje Intermedio** o **IL** (sigla en inglés de **Intermediate Language**), es un lenguaje muy cercano al assembler, pero es Independiente del Código Máquina de cualquier Computadora. Un **IL** es un **lenguaje inventado** por alguna persona, que imagina que su Lenguaje Intermedio será soportado (entendido) por una Máquina (computadora) Abstracta (imaginaria). A esta “computadora imaginaria”, formalmente, se le llama **Máquina Virtual** o **Virtual Machine** (cuya abreviatura es **VM**).

\*\* **Código Intermedio** = Programa escrito en un Lenguaje Intermedio.

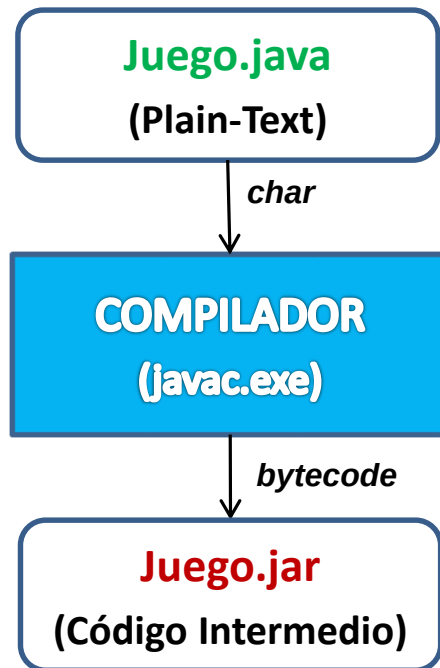
# 1.2. Compilación a Código Intermedio

7

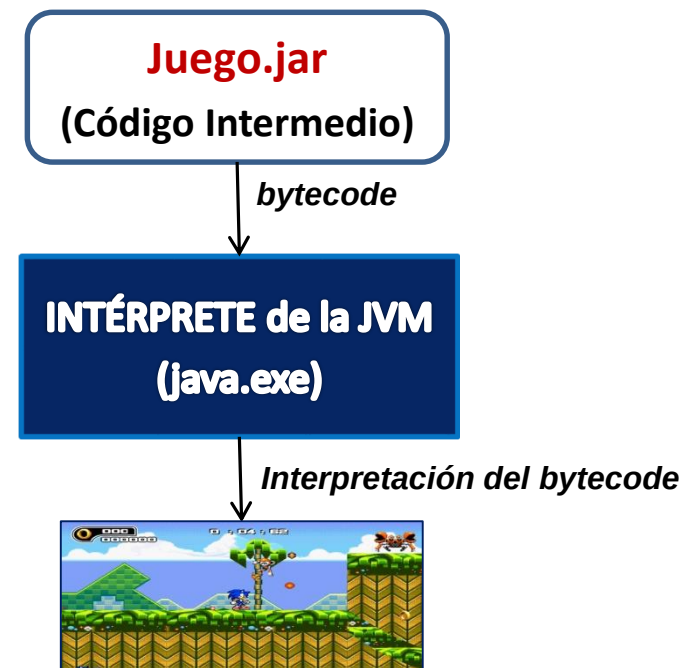
El lenguaje actual más famoso en esta práctica es JAVA, pues su compilador **javac.exe** traduce el Programa Fuente (**.java**) a un Programa Objeto (**.jar**) escrito en un **IL**, donde cada instrucción de éste IL se representa por una secuencia de bytes llamada *bytecode*.

La ejecución (run) del **.jar** la realiza el programa **java.exe**, el cual es el Intérprete de la Java Virtual Machine o JVM.

## Compilación en Java



## Ejecución en Java



# 1.3. El Usuario de un Compilador.

Cuando una persona desea compilar un programa, tiene normalmente dos opciones:

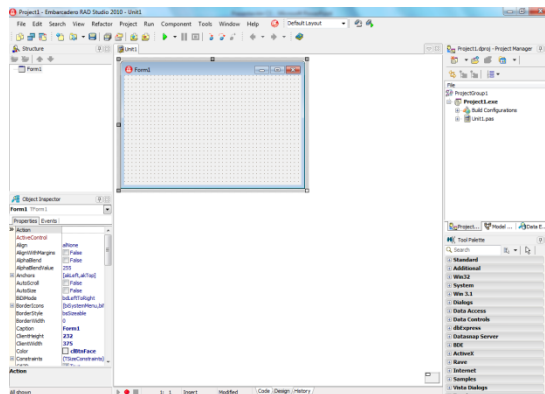
- Usar directamente al **Compilador**, pasándole el **Programa Fuente** desde la línea de comandos (cmd). Por ejemplo para los compiladores DELPHI (dcc32.exe) y JAVA (Javac.exe), respectivamente:

```
C:\Program Files\Embarcadero\RAD Studio\7.0\bin>dcc32.exe MiPrograma.pas
```

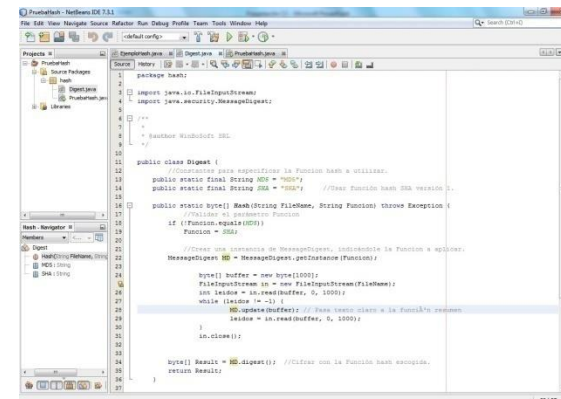
```
C:\>javac MiPrograma.java
```

- Usar cómodamente un **IDE**, el cual no solo se encargará de **interactuar** con el **Compilador**, sino que también le permitirá abrir/guardar/editar archivos, detectar errores previos, depurar, ejecutar, etc. Para DELPHI, por ejemplo, se destaca el IDE de Embarcadero (bds.exe) y para JAVA: NetBeans, Eclipse, IntelliJ Idea, Android Studio .

**Embarcadero RAD Studio (IDE DELPHI)**

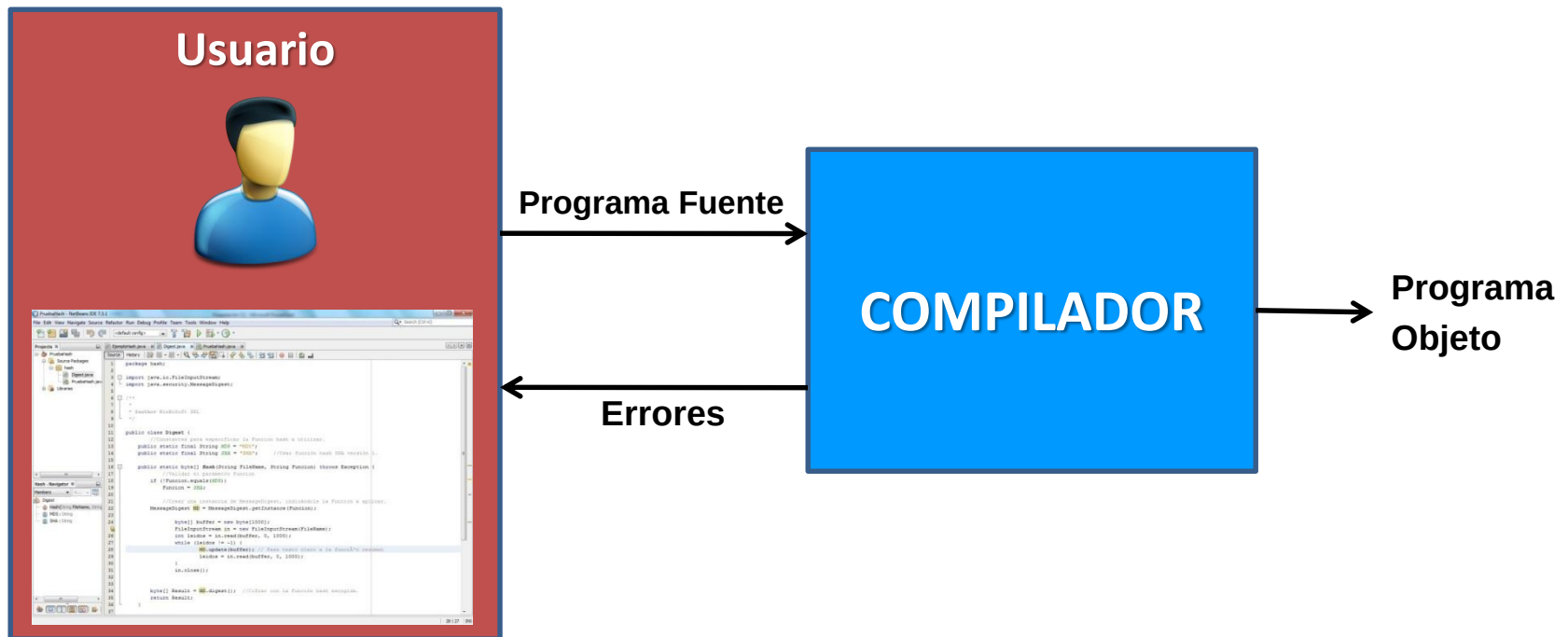


**NetBeans (IDE JAVA)**



## 1.3. El Usuario de un Compilador.

Entonces, normalmente, el **Usuario** de un **Compilador** es el **Programador** (una persona) o el **IDE\*** (un software). El usuario envía el **Programa Fuente** al **Compilador** y si no hay errores lo convierte a **Código Objeto**; pero, si el **Compilador** encuentra **Errores**, se lo comunica al **Usuario**.



\*De Wikipedia: “Un **Entorno de Desarrollo Integrado**, llamado también **IDE** (sigla en inglés de **Integrated Development Environment**), es un programa informático compuesto por un conjunto de herramientas de programación: Editor de Texto, Compilación -Interacción con un Compilador-, Ejecución, Depuración (Debug), etc. Puede dedicarse en exclusiva a un solo lenguaje de programación o bien puede utilizarse para varios”.

## 1.4. Tipos de Compiladores.

Por su accionar (*es decir, la forma en que se comportan con el Programa Fuente y con la traducción*), los Compiladores se dividen en varios tipos. Aquí destacamos seis de ellos: **Compiladores de una o varias pasadas**, **Compiladores Cruzados**, **Compiladores Nativos**, **Compiladores Optimizadores**, **Compiladores JIT** (Just in Time), **Metacompilador**.

Debe recordarse que esta taxonomía de los Compiladores no es excluyente, por lo que puede haber **Compiladores** que se adscriban a **varios tipos**.

### COMPILADOR DE UNA O VARIAS PASADAS

A veces un Compilador, por la complejidad del Lenguaje Fuente, necesita leer 'más de una vez al Programa Fuente; si esto ocurre, se dice que el Compilador es un **Compilador de Varias o Múltiples Pasadas** (Multi-pass Compiler). Por ejemplo, muchos Ensambladores, como el MASM de Microsoft, es un Compilador de Varias Pasadas.

Por otro lado, si el Compilador lee una sola vez el Programa fuente, se dice que es un **Compilador de Una Pasada** (One-pass Compiler). Por ejemplo, el Compilador de JAVA, es un Compilador de Una Pasada.

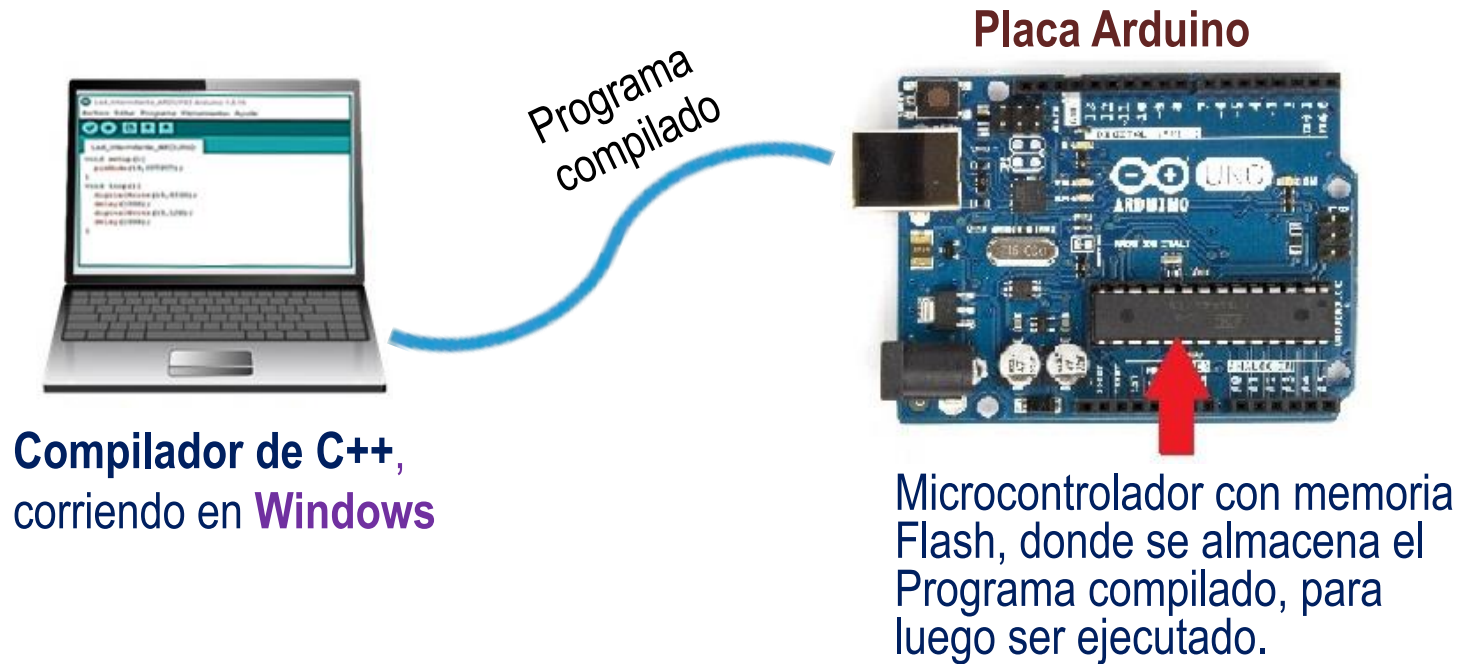
## 1.4. Tipos de Compiladores.

### COMPILADOR CRUZADO

Un **Compilador Cruzado** (Cross-Compiler), es aquel que genera código ejecutable para una plataforma diferente a aquella en la que se ejecuta. Esta herramienta es útil cuando quiere compilarse código para una plataforma a la que no se tiene acceso o cuando es incómodo o imposible compilar en dicha plataforma (*como en el caso de los sistemas empotrados: Arduino, Raspberry Pi, etc.*)

Por ejemplo, si queremos desarrollar un programa C++ para una placa de Arduino, este debe ser escrito y traducido por un Compilador en una PC. Luego el programa compilado debe ser enviado a la tarjeta Arduino para su ejecución.

## 1.4. Tipos de Compiladores.



Claramente, el **Compilador C++** es un *compilador cruzado*, porque él corre en una Plataforma (**PC + Windows**) distinta a la Plataforma (**Arduino**) a la cual traduce (compila) el Programa Fuente C++.

## 1.4. Tipos de Compiladores.

### Otro ejemplo de Compilador Cruzado:

Supongamos que queremos desarrollar una Aplicación (App), usando Java, para un **Teléfono Móvil con Android**. En este caso, no escribiremos el Código Fuente Java de la App en el teléfono, sino más bien en una PC (tal vez usando Android Studio en una **PC con Windows**).

Una vez el **Compilador JAVA** genera el Código Objeto en un archivo con extensión **.apk**, copiamos este archivo al teléfono móvil, para ser ejecutado.

Así, el **Compilador JAVA** es un *compilador cruzado*, porque él se ejecuta en una Plataforma (**PC + Windows**) distinta a la Plataforma (**Teléfono + Android**) a la cual traduce (compila) el Programa Fuente JAVA.

## 1.4. Tipos de Compiladores.

### COMPILADOR NATIVO

En contraste a los *compiladores cruzados*, los **Compiladores Nativos**, convierten el código fuente a código máquina que es específico para la computadora en la cual se ejecuta el compilador. Esto significa que el archivo ejecutable generado está optimizado para el sistema operativo y la arquitectura de hardware de la máquina donde se realizó la compilación.

Por ejemplo, si desarrollamos un programa Prog.pas para Delphi y compilamos el mismo con un compilador de **DELPHI para Windows**, obtendremos un archivo ejecutable (stand-alone) Prog.exe, el cual correrá sin ningún problema en el SO **Windows**.

(El compilador de **DELPHI** es **nativo** de **Windows**, pues él corre en **Windows** y genera un ejecutable –archivo.exe– para **Windows**)

## 1.4. Tipos de Compiladores.

### COMPILADOR OPTIMIZADOR

Un compilador optimizador es aquel que trata de minimizar ciertos atributos de un programa con el fin de aumentar la eficiencia y rendimiento. Esta *eficiencia y rendimiento* se logra:

- Reduciendo el tiempo de ejecución del Programa Objeto
- Reduciendo la cantidad de memoria que ocupa el Programa Objeto en ejecución.
- Reduciendo el tamaño del Programa Objeto.

Actualmente, muchos Compiladores profesionales (JAVA, DELPHI, etc) usan la optimización de código.

Debe recordarse que la Optimización, se realiza después de la Generación de Código. Este tema se aborda en la (siguiente) sección 2.

## 1.4. Tipos de Compiladores.

### COMPILADOR JIT (Just in Time)

Un **Compilador JIT**, Just-in-Time, o Compilador Dinámico (*o sea en tiempo de ejecución*), es aquel Compilador que es usado cuando el **Programa Objeto**, que es un **Código Intermedio**, está corriendo (run).

El Compilador JIT entra en acción, traduciendo a Código Máquina nativo, funciones o un archivo completo del Código Intermedio. En el caso de una función, el código correspondiente a esta función, se compila justo cuando va a ejecutarse (*de aquí el nombre de just-in-time, “justo a tiempo”*).

### METACOMPILADOR

El **Metacompilador** es sinónimo de **compilador de compiladores** y se refiere a un programa que recibe como entrada las especificaciones del lenguaje para el que se desea obtener un compilador y genera como salida el compilador para ese lenguaje. El desarrollo de los metacompiladores se encuentra con la dificultad de unir la generación de código con la parte de análisis. Lo que sí se han desarrollado son generadores de analizadores léxicos y sintácticos.

## 2. Las Fases de un Compilador.

Un Compilador, lleva a cabo su tarea de traducción en dos etapas: la **etapa de análisis** y la **etapa de síntesis**.

En la primera etapa, la de **análisis**, el compilador lee caracter (char) a caracter el Programa Fuente F, con la intención de verificar su correctitud: Si encuentra errores en F, el compilador comunica al usuario la presencia y ubicación de los mismos y **finaliza** su ejecución; Si no hay errores, continúa con la siguiente etapa.

A no haber errores en la primera etapa, el Compilador ingresa a su segunda etapa: la etapa de **síntesis**. En esta etapa es donde el Compilador realmente traduce al Programa Fuente **F** en un Programa Objeto **O**.

# 2. Las Fases de un Compilador.

Cada etapa a su vez se subdivide, cada una, en tres **fases**:



Por lo tanto, un compilador lleva a cabo **SEIS** fases para realizar la traducción de **F** a **O**.

Sin embargo, las fases son ayudadas por dos módulos adicionales: La **Tabla de Símbolos** y el **Manejador de Errores**, tal como se muestra a continuación ...

## 2. Las Fases de un Compilador.

19



## 2.1. La Tabla de Símbolos

Un **identificador** o **identifier** -abreviado **ID**-, es un nombre que el programador libremente le otorga a un elemento del lenguaje (class, procedimiento, función, variable, etc).

Por ejemplo, en este fragmento de código:

```
public class PilaX {  
    private boolean Sw;  
    String Modo(int x) {  
        ...  
    }  
}
```

Los **ID**'s son **PilaX**, **Sw**, **Modo** y **x**.

**Nota.** No se debe confundir un **ID** con una **String Constante** (también llamada **Literal**). Estas últimas están encerradas entre **comillas** y normalmente son usadas para mensajes. Por ejemplo, **"Hola Mundo"**, es una **String Constante**.

## 2.1. La Tabla de Símbolos

Una **Tabla de Símbolos (TS)** es una **Estructura de Datos** que usa un Compilador o un Intérprete, en el proceso de traducción, en donde cada símbolo del Código Fuente (variable, constante, procedimiento, class, ...) está asociado con información tal como la ubicación, el tipo de datos y el ámbito -scope- al cual pertenece.

Entonces en una **TS** se almacenan **todos** los **identificadores (ID)** del programa y **otra** información que el Compilador o Intérprete juzguen necesarios.

Los datos contenidos en la **TS** son utilizados por las seis fases del Compilador. Normalmente –como se observa en el [Diagrama de Fases](#)– el Analizador Léxico y el Analizador Sintáctico escriben/leen en ella, mientras que las demás fases solo leen sus datos.

Aunque a la **TS** se la menciona en singular, en realidad una Tabla de Símbolos, normalmente, está compuesta por **varias Tablas**, según sea la necesidad del Compilador o Intérprete.

## 2.1. La Tabla de Símbolos

Por ejemplo, en la figura mostrada, se muestra la **TS** de un pequeño Compilador, compuesta por dos tablas:

- La primera tabla mantiene la información de los procedimientos y funciones.
- La segunda tabla almacena las variables. Nótese que en la misma, cada variable informa sobre el ámbito (scope) al cual pertenece: Por ejemplo, la variable “k” pertenece a la función “Factorial” y “Altura” al procedimiento “\$Main”.

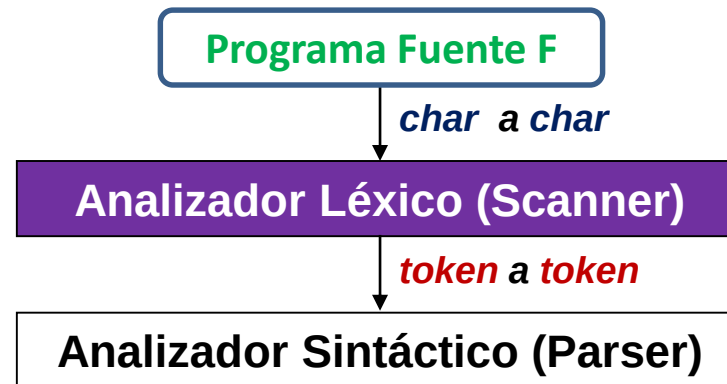
| Index    | NOMBRE      | CLASE     | TIPO    | Inicio IL | PARÁMETROS |
|----------|-------------|-----------|---------|-----------|------------|
| <b>0</b> | “Factorial” | FUNCTION  | INTEGER | 20        | 1          |
| 1        | “Linea”     | PROC      | –       | 10        | 0          |
| <b>2</b> | “isValido”  | FUNCTION  | BOOLEAN | 30        | 1          |
| <b>3</b> | “\$Main”    | MAIN_PROC | –       | 0         | 2          |

| NOMBRE   | TIPO    | SCOPE    |
|----------|---------|----------|
| “k”      | INTEGER | <b>0</b> |
| “sw”     | BOOLEAN | <b>2</b> |
| “MAX”    | INTEGER | GLOBAL   |
| “Altura” | INTEGER | <b>3</b> |

**Nota.** La **TS** mostrada aquí es solamente para ejemplificar. En la práctica, una TS puede tener tablas que utilicen Estructuras de Datos más complejas (e.g. Hash Tables); todo depende de los requerimientos y del diseño del Compilador.

## 2.2. El Analizador Léxico

Tal como se observa en el [Diagrama de Fases](#), el **Analizador Léxico**, también conocido como **Scanner**, **Lexer** o **Analex**, es la primera fase del Compilador. Recibe datos del Programa Fuente y envía información al Analizador Sintáctico (**Parser**).



El **Analizador Léxico**, lee **uno a uno** los caracteres (**char's**) del **Programa Fuente** con la intención de *reconocer* **substrings** –que obedecen a un patrón de formación–, llamados **token's**. Los **token's** son enviados uno por uno al **Parser**, donde él los tratará como una **única entidad**.

## 2.2. El Analizador Léxico

Por ejemplo, si el **Programa Fuente F** es

**F** = "x := 640 \* Velocidad - 456;"

el **Analex** agrupará **caracteres consecutivos (substrings)** para formar **token's** :

|           |               |            |            |                  |              |            |                |
|-----------|---------------|------------|------------|------------------|--------------|------------|----------------|
| <b>x</b>  | <b>:=</b>     | <b>640</b> | <b>*</b>   | <b>Velocidad</b> | <b>-</b>     | <b>456</b> | <b>;</b>       |
|           |               |            |            |                  |              |            |                |
| <b>ID</b> | <b>ASSIGN</b> | <b>NUM</b> | <b>POR</b> | <b>ID</b>        | <b>MENOS</b> | <b>NUM</b> | <b>PTOCOMA</b> |

Nótese que los **token's** tienen un nombre o abreviatura asignada (ASSIGN, POR, MENOS, etc). Estos nombres usualmente son "inventados" por el diseñador del compilador, aunque a los tokens más conocidos, como **ID** (identifier) y **NUM** (number), normalmente se les respeta la abreviatura *tradicional* que tienen.

## 2.2. El Analizador Léxico

- Se debe puntualizar que el **Analex**, no verifica el orden en que deben aparecer los **tokens**. Su trabajo solo se circunscribe a **reconocerlos**. Por ejemplo, si el Programa Fuente (erróneo) **F** fuera

```
F = "x - := 640 456; Velocidad *"
```

el **Analex** reconocería uno a uno los **tokens**, en este orden: ID, MENOS, ASSIGN, NUM, NUM, PTOCOMA, ID, POR, sin comunicar **ningún** error.

- La fase del **Análisis Léxico**, es considerada la fase **más lenta** del Compilador, porque lee char a char al Programa Fuente.
- Aparte de reconocer los tokens, el **Analex** también hace tareas adicionales: Ignorar los **comentarios** del Programa Fuente e instalar (insertar) las **StringCtte's** e **ID's** en la TS (si corresponde).

## 2.3. El Analizador Sintáctico

En el diccionario, se define **sintaxis**, como la “*parte de la gramática que estudia la **ordenación** y las relaciones mutuas de las palabras en la oración, hasta formar una expresión completa*”.

En compilación, la **sintaxis** es el “*conjunto de reglas (gramática) que definen las secuencias correctas (**orden**) de los elementos (**tokens**) de un lenguaje de programación*”.

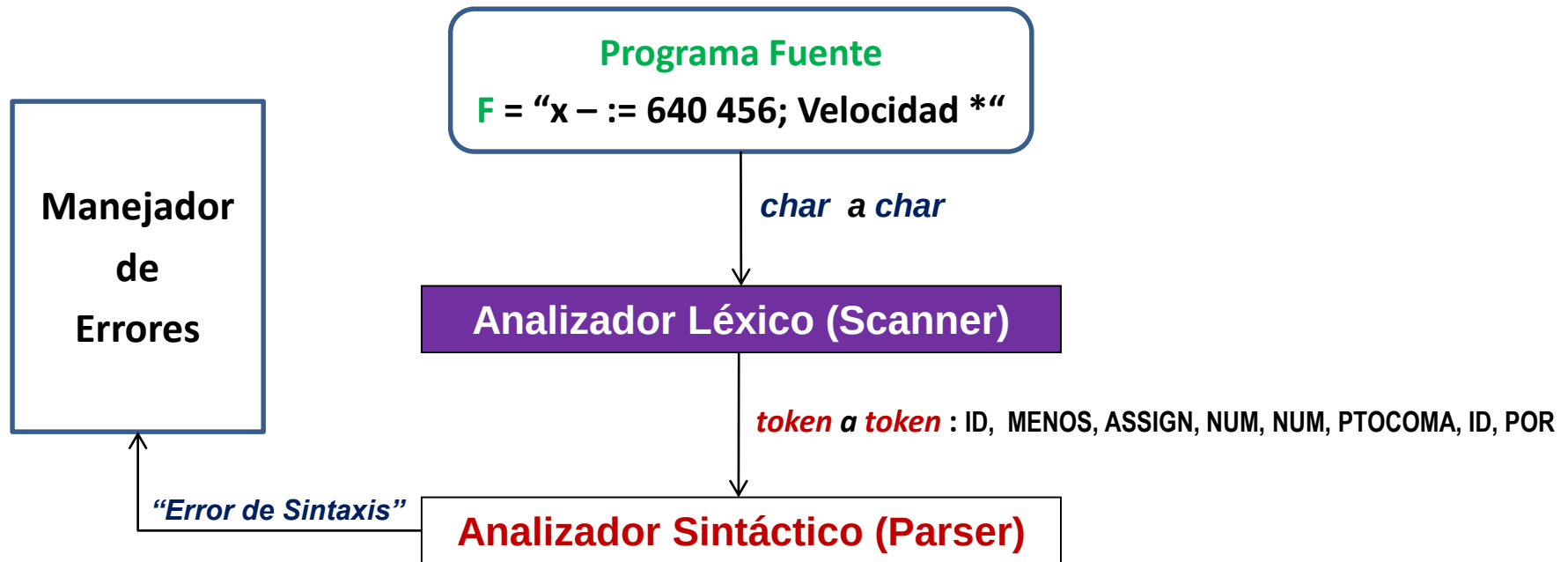
Entonces,

El **Analizador Sintáctico**, más conocido como **Parser**, es la fase del Compilador que revisa el **orden correcto** en que deben aparecer los **tokens** en el programa fuente.

## 2.3. El Analizador Sintáctico

27

Por ejemplo:



Aquí, vemos el proceder del **Analizador Sintáctico**: El **Scanner** envía los **tokens** al **Parser** y éste último, al verificar que éstos no están en el **orden** correcto, emite un error.

## 2.4. El Analizador Semántico

En la lingüística, el término **semántica** se refiere a los aspectos del **significado**, sentido o interpretación de un determinado elemento, símbolo, palabra, expresión o representación formal.

En Compilación, entendemos **semántica** como la interpretación del **significado de las sentencias** escritas en el **Programa Fuente** (¿Qué desea hacer el programador con la sentencia escrita?).

Por ejemplo, consideremos la siguiente sentencia escrita en C++:

$$s = a + b;$$

si las variables son del tipo *string*, significará una *concatenación*;  
sin son **enteras** (int) se interpretará como una **suma**.

## 2.4. El Analizador Semántico

Por tanto, en definitiva, el **Analizador Semántico** es el encargado de comprobar que el **significado** de lo que se va leyendo es válido. Esto implica, entonces, que esta fase se dedicará a comprobar la compatibilidad de los tipos de datos, los rangos de una asignación, etc.

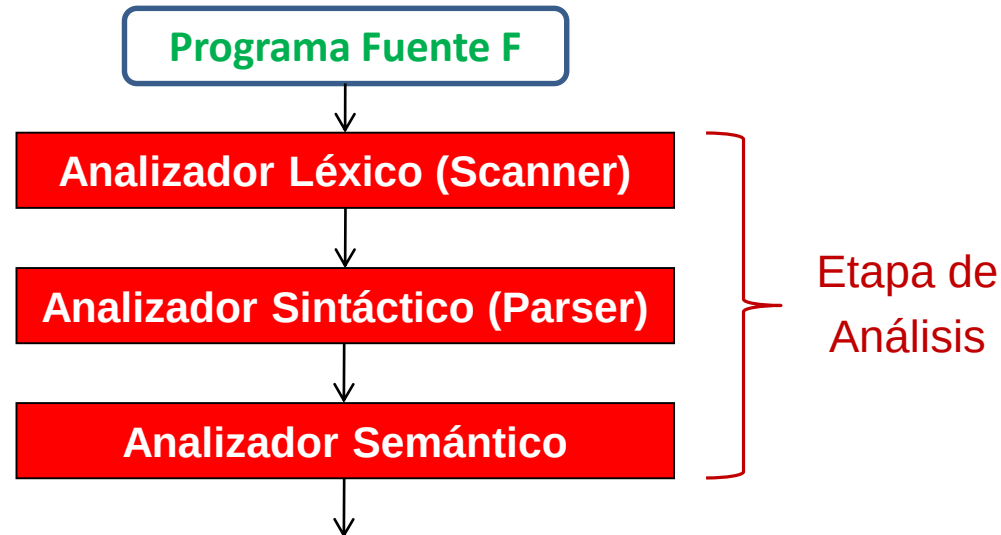
Por ejemplo, en el siguiente fragmento de **Código** (sintácticamente bien formado), ésta fase encontrará **errores semánticos**:

```
byte x, y;  
string s;  
int V[5];  
x = 300;           //Out of range: byte va de 0 a 255  
y = s + x;         //Tipos incompatibles: (s es string)  
V[100] = 0;        //Out of range: No existe la casilla 100.
```

## 2.5. El Generador de Código Intermedio

30

Tal como establece el [Diagrama de Fases](#), una vez **concluida** la **Etapa de Análisis**



y al constatar que el **Programa Fuente F** es **válido** (no tiene errores), el Compilador pasa a la **primera fase** de la **Etapa de Síntesis**, llamada **Generación de Código Intermedio**.

## 2.5. El Generador de Código Intermedio

31

El **Generador de Código Intermedio** convierte al **Programa Fuente F** en un Código muy cercano al assembler, llamado lenguaje intermedio (**IL**, Intermediate Language), con la intención de facilitar el trabajo de las siguientes dos fases (Optimación de Código y Generación de Código Objeto final) de la **Etapas de Síntesis**.

Por ejemplo, el programa fuente

```
x := 640 * Velocidad - 456;
```

se convertirá en el Código Intermedio

```
t1 = 640
t2 = Velocidad
t3 = t1 * t2
t4 = 456
t5 = t3 - t4
x = t5
```

**Nota.** El **IL** (lenguaje intermedio) mostrado aquí es llamado “Código de Tres Direcciones” y es considerado el más “popular” de todos los **IL**’s.

Sin embargo, el diseñador del Compilador puede optar por otro **IL** disponible, o uno de su propia creación.

## 2.6. El Optimador de Código.

32

El **Optimador de Código Intermedio**, es la fase encargada de **reducir** el tamaño del **Código Intermedio** con la finalidad de que el **Código Objeto** final sea más pequeño y más rápido.

Por ejemplo:

```
t1 = 640
t2 = Velocidad
t3 = t1 * t2
t4 = 456
t5 = t3 - t4
x = t5
```

OPTIMACIÓN

```
t1 = 640
t3 = t1 * Velocidad
t4 = 456
x = t3 - t4
```

## 2.7. El Generador de Código Objeto Final.

33

Una vez optimizado (reducido) el **Código Intermedio**, esta fase genera el **Código Objeto Final**.

Por ejemplo:

```
t1 = 640  
t3 = t1 * Velocidad  
t4 = 456  
x = t3 - t4
```

**Generar  
Código Objeto**

### **Código Objeto Final**

```
MOV [t1], 640  
MOV AX, Velocidad  
MUL BX  
MOV [t3], BX  
..... (etc)
```

### 3. ¿Qué es un Intérprete?

Un **intérprete** o interpretador es un software capaz de **analizar** y **ejecutar** (run) otros programas, escritos en un Lenguaje de Programación.

*Los intérpretes sólo realizan la traducción a medida que sea necesaria, típicamente, instrucción por instrucción, y normalmente **no guardan** el resultado de dicha traducción.*

## 3.1. Un Ejemplo Sencillo.

Para entender mejor los conceptos vertidos en la diapositiva anterior, supóngase que usted realiza un software (el cual será un **Intérprete**), que inicialmente tiene a un cuadrado dibujado en el centro de la pantalla, y que acepta como instrucciones (sentencias u órdenes) de animación del cuadrado los siguientes caracteres:

- '-' Mover el cuadrado una posición a la izquierda.
- '+' Mover el cuadrado una posición a la derecha.
- '\*' Mover el cuadrado una posición hacia arriba.
- '/' Mover el cuadrado una posición hacia abajo.

Entonces, el **usuario** de su software, podrá escribir un “Programa Fuente” **F** para su **Intérprete**, simplemente almacenando en un string o archivo texto **F** una secuencia de órdenes.

## 3.1. Un Ejemplo Sencillo.

Por ejemplo, si **F** = “+\*\*--/+”, su **usuario** quiere mover el cuadrado una posición a la derecha dos hacia arriba, dos hacia la izquierda una hacia abajo y finalmente una posición hacia la derecha. Claramente, para **interpretar** a **F**, usted tendría un procedimiento parecido a éste

```
void Interpretar(String F) {  
    for (int i=0; i<F.length(); i++) { //Para leer cada “sentencia” de F.  
        char orden = F.charAt(i); //Obtener el char de la pos. i de F.  
        switch (orden) { // Interpretar la orden.  
            case '+' : Cuadrado.MoverDerecha(); break;  
            case '-' : Cuadrado.MoverIzquierda(); break;  
            case '*' : Cuadrado.MoverArriba(); break;  
            case '/' : Cuadrado.MoverAbajo(); break;  
            default : println("Error en Sentencia");  
        }  
    }  
}
```

# 3.1. Un Ejemplo Sencillo.

Entonces, su **Intérprete** luego de ejecutar al Programa Fuente **F**, habrá recibido una serie de 7 instrucciones y por cada una de ellas habrá emitido una **traducción**:

## Instrucción u Orden recibida

+

\*

\*

-

-

/

+

## Traducción Efectuada

Cuadrado.MoverDerecha()

Cuadrado.MoverArriba()

Cuadrado.MoverArriba()

Cuadrado.MoverIzquierda()

Cuadrado.MoverIzquierda()

Cuadrado.MoverAbajo()

Cuadrado.MoverDerecha()

Estas son las **traducciones efectuadas** por el **intérprete** y **no son guardadas** en ninguna parte (archivo, base de datos, etc).

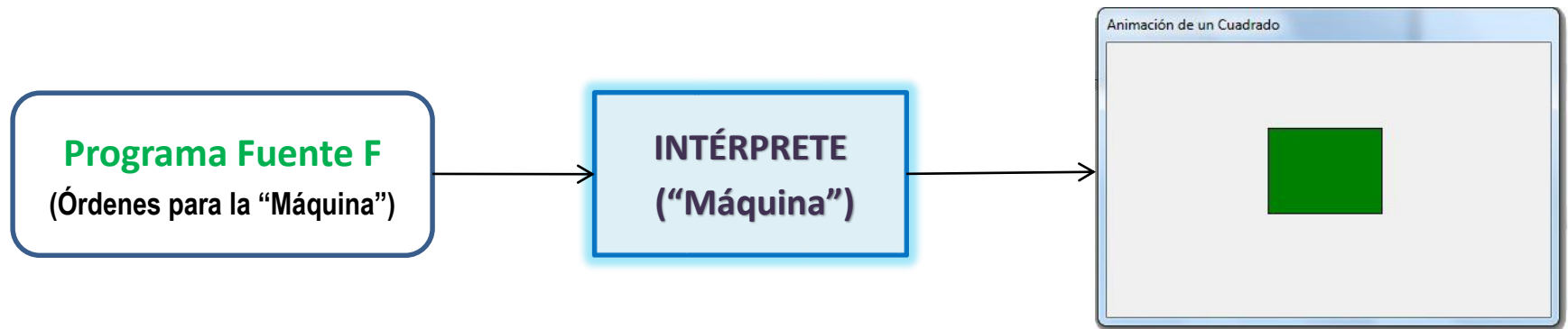


# 3.1. Un Ejemplo Sencillo.

Si razonamos un poco, tal vez podamos afirmar que desde la perspectiva del usuario, al **Intérprete** él lo podrá ver como:

**Runner** o **Ejecutor**: Porque ejecuta (run) las instrucciones que el usuario anota en el Programa Fuente **F**.

**Máquina** (Abstracta): Porque al software lo podrá observar como una Máquina (imaginaria) que anima un cuadrado según las instrucciones u órdenes dadas en **F**.



## 3.2. Diferencia entre Compilador e Intérprete

La diferencia es bastante simple de detectar:

Un **Compilador**, *solo convierte* un Programa (**Fuente**) en otro Programa (**Objeto**) equivalente. Un Compilador **NO ejecuta** ninguna instrucción del Programa Fuente, ni del Programa Objeto.

En cambio, un **Intérprete** luego de analizar al Programa **Fuente**, **ejecuta** cada una de sus Instrucciones. Un Intérprete **NO genera** un Programa **Objeto** equivalente al Fuente.

### 3.3. Ventajas y Desventajas de un Intérprete

Los **Programas Fuente interpretados** suelen ser más lentos que los compilados debido a la necesidad de traducir el programa mientras se ejecuta, pero a cambio son más flexibles como entornos de programación y depuración (debug). Lo que implica, por ejemplo, en una mayor facilidad para reemplazar partes enteras del programa o añadir módulos completamente nuevos.

Permiten ofrecer al **Programa** interpretado un entorno no dependiente de la Plataforma donde se ejecuta el Intérprete, sino del propio **Intérprete** (una razón más para llamarlo “Máquina”).

# 1.1. Compilador Estándar y No-Estándar

Desde sus inicios, los Compiladores fueron creados para leer **Programas Fuentes** escritos en Plain-Text y generar **Programas Objetos** *stand-alone* de Código Máquina absoluto, a los cuales le llamamos “ejecutables”.

A éste tipo de Compiladores, se les llama **Compiladores Estándar o Clásicos**.

Por ejemplo, en el Diagrama de la derecha, se observa un **Compilador Estándar** que lee un Programa Fuente de C++ y lo convierte a un archivo “ejecutable” en el formato de Windows.

